



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №9
з дисципліни
“Технології розроблення програмного забезпечення”

Варіант №7
“Редактор зображень”

Виконав
студент групи ІА–22:
Гордун Д.В.

Перевірив
Мягкий М.Ю.

Київ 2024

Тема: РІЗНІ ВИДИ ВЗАЄМОДІЇ ДОДАТКІВ: CLIENT-SERVER, PEER-TO-PEER, SERVICE-ORIENTED ARCHITECTURE

Мета: Ознайомитися з різними видами взаємодії додатків. Реалізувати частину функціоналу робочої програми у вигляді класів і їх взаємодій для досягнення конкретних функціональних можливостей. Реалізувати взаємодію програми в одній з архітектур відповідно до обраної теми.

Хід роботи

Додав код за посиланням:

<https://github.com/DmytroH25/trpz/pull/8/commits/7a490b0556451f634b3b4ef65f489b4671af3889>

Короткі теоретичні відомості

Клієнт-серверні додатки



Клієнт-серверні додатки являють собою найпростіший варіант розподілених додатків, де виділяється два види додатків: клієнти (представляють додаток користувачеві) і сервери (використовується для зберігання і обробки даних). Розрізняють тонкі клієнти і товсті клієнти.

Тонкий клієнт - клієнт, який повністю всі операції (або більшість, пов'язаних з логікою роботи програми) передає для обробки на сервер, а сам зберігає лише візуальне уявлення одержуваних від сервера відповідей. Грубо кажучи, тонкий клієнт - набір форм відображення і канал зв'язку з сервером.

У такому варіанті використання дуже велике навантаження лягає на сервер. Раніше такий варіант був доступний, оскільки обчислень було небагато, термінали були дорогими і як правило йшли з урізаним обчислювальним пристроєм. У нинішній час зі зростанням обчислювальних можливостей клієнтських машин має сенс частину обчислень перекладати на клієнтські комп'ютери.

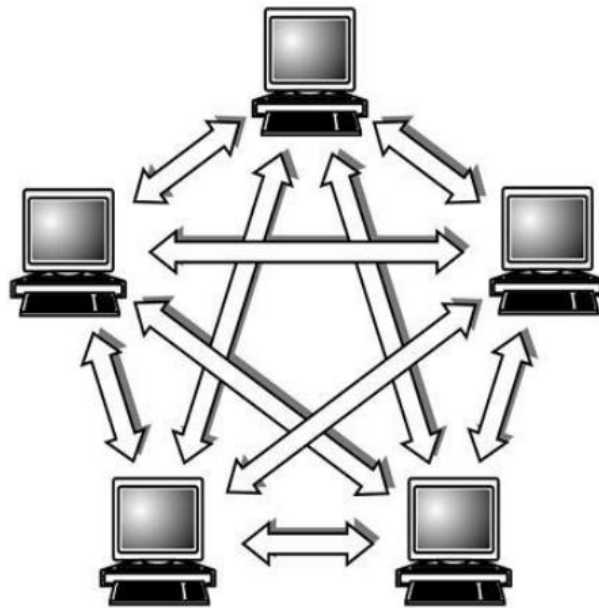
Однак дана модель має сенс ще в захищених сценаріях - коли зайві дані не можна показувати клієнтським комп'ютерам в зв'язку з небезпекою взлому. Ще однією проблемою може бути множинний доступ - щоб уникнути конфліктності даних має сенс централізація обчислень в одному місці (на сервері) для визначення послідовності операцій та уникнення пошкодження даних.

Товстий клієнт - антипод тонкого клієнта, більшість логіки обробки даних містить на стороні клієнта. Це сильно розвантажує сервер. Сервер в таких випадках зазвичай працює лише як точка доступу до деякого іншого ресурсу (наприклад, бази даних) або сполучна ланка з іншими клієнтськими комп'ютерами.

У моделі клієнт-серверної взаємодії також важливе місце займає модель «підписки / видачі». Комп'ютери клієнтів можуть «підписуватися» серверу на певний набір подій (поява нових користувачів, зміна даних тощо), а сервер в свою чергу сповіщає клієнтські комп'ютери про походження цих подій. Даний спосіб взаємодії реалізується за допомогою шаблону «оглядач»; він несе велике навантаження на сервер (необхідно відстежувати хто на що підписався) і на канали зв'язку (багато повторюваних даних відправляються на різні точки в мережі). З іншого боку, це дуже зручно для клієнтських машин для організації оновлення даних без їх повторного перезапросу. Природно, інші механізми включають ліниву ініціалізацію, перезапрос, запит за таймером, запит за потребою (після натискання кнопки «оновити» наприклад).

Клієнт-серверна взаємодія, як правило, організовується за допомогою 3-х рівневої структури: клієнтська частина, загальна частина, серверна частина. Оскільки велика частина даних загальна (класи, використовувані системою), їх прийнято виносити в загальну частину (middleware) системи. Клієнтська частина містить візуальне відображення і логіку обробки дії користувача; код для встановлення сеансу зв'язку з сервером і виконання відповідних викликів. Серверна частина містить основну логіку роботи програми (бізнес-логіку) або ту її частину, яка відповідає зберіганню або обміну даними між клієнтом і сервером або клієнтами.

Додатки типу «peer-to-peer»



Модель взаємодії додатків типу peer-to-peer має на увазі рівноправ'я клієнтських програм і відсутність серверної програми. Усі клієнтські програми контактують між собою для виконання спільних цілей. У таких мережах найчастіше виникають наступні проблеми: синхронізація даних, пошук клієнтських застосувань.

Для пошуку клієнтських застосувань може використовуватися одна загальна адреса, що містить набір адрес підключених клієнтів. Це в деякому роді нагадує сервер. Проте сервер є лише центром сходження безлічі клієнтів, а адреси можуть зберігатися в самих клієнтських застосуваннях. Це називається структуровані однорангові мережі.

Для вирішення проблеми синхронізації даних використовуються традиційні підходи: можливий варіант договору про набір для синхронізації (negotiation), або визначення по деякому алгоритму порівняння даних (hash-алгоритми і інші). Як правило, peer - to - peer застосування є звичайними застосуваннями з продуманою інфраструктурою обміну повідомленнями між клієнтами. Для таких випадків як правило розробляються спеціальні формати і протоколи обміну для структуризації діалогу між клієнтами.

Сервіс-орієнтована архітектура.

Сервіс-орієнтована архітектура (SOA, англ. service-oriented architecture) — модульний підхід до розробки програмного забезпечення, заснований на використанні розподілених, слабо пов'язаних (англ. Loose coupling) замінних компонентів, оснащених стандартизованими інтерфейсами для взаємодії за стандартизованими протоколами.

Програмні комплекси, розроблені відповідно до сервіс-орієнтованою архітектурою, зазвичай реалізуються як набір веб-служб, взаємодіючих за протоколом SOAP, але існують і інші реалізації (наприклад, на базі jini, CORBA, на основі REST).

Інтерфейси компонентів в сервіс-орієнтованій архітектурі інкапсулюють деталі реалізації (операційну систему, платформу, мову програмування) від інших компонентів, таким чином забезпечуючи комбінування і багаторазове використання компонентів для побудови складних розподілених програмних комплексів, забезпечуючи незалежність від використовуваних платформ та інструментів розробки, сприяючи масштабованості і керованості створюваних систем.

Реалізація частину функціоналу робочої програми у вигляді класів і їх взаємодій для досягнення конкретних функціональних можливостей.

com.example.imageeditor.config

- **DataLoader:** Ініціалізує базу даних з ролями за замовчуванням та адміністративним користувачем під час запуску додатка.

com.example.imageeditor.controller

- **ImageController:** Обробляє HTTP-запити, пов'язані з операціями над зображеннями, такими як завантаження, обертання, зміна розміру, обрізка, скасування, повторення та завантаження.

com.example.imageeditor.dto

- **LoginRequestDto:** Об'єкт передачі даних (DTO) для запитів на вхід.
- **LoginResponseDto:** DTO для відповіді на запити на вхід.
- **RegisterRequestDto:** DTO для запитів на реєстрацію користувачів.

com.example.imageeditor.facade

- **ImageFacade:** Надає спрощений інтерфейс для операцій із зображеннями, делегуючи їх відповідним сервісам.

com.example.imageeditor.repository

- **ImageRepository:** Інтерфейс репозиторію для роботи із сутностями Image.
- **RoleRepository:** Інтерфейс репозиторію для роботи із сутностями Role.
- **UserRepository:** Інтерфейс репозиторію для роботи із сутностями User.

com.example.imageeditor.repository.entity

- **Image:** Сутність, яка представляє завантажене зображення.
- **Role:** Сутність, яка представляє роль користувача (наприклад, ROLE_USER, ROLE_ADMIN).
- **User:** Сутність, яка представляє користувача додатка.

com.example.imageeditor.service

- **AuthService:** Інтерфейс для операцій, пов'язаних з автентифікацією.
- **ImageService:** Інтерфейс для операцій над зображеннями.
- **UserService:** Інтерфейс для операцій, пов'язаних із користувачами.

com.example.imageeditor.service.impl

- **AuthServiceImpl:** Реалізація інтерфейсу AuthService.
- **ImageServiceImpl:** Реалізація інтерфейсу ImageService.
- **ServiceImpl:** Реалізація інтерфейсу UserService.
- **UserDetailsServiceImpl:** Реалізація інтерфейсу UserDetailsService для завантаження даних користувача.

com.example.imageeditor.service.exception

- **ImageNotFoundException:** Користувацьке виключення, яке викидається, якщо зображення не знайдено.
- **RoleNotFoundException:** Користувацьке виключення, яке викидається, якщо роль не знайдено.
- **UserNotFoundException:** Користувацьке виключення, яке викидається, якщо користувача не знайдено.

com.example.imageeditor.state

- **ImageState:** Інтерфейс для застосування ефектів до зображень.
- **RotateState:** Реалізація інтерфейсу ImageState для обертання зображень.
- **CropState:** Реалізація інтерфейсу ImageState для обрізки зображень.
- **ResizeState:** Реалізація інтерфейсу ImageState для зміни розміру зображень.

com.example.imageeditor.util

- **JwtUtils:** Утилітний клас для генерації та перевірки JWT-токенів.

com.example.imageeditor.composite

- **ImageComponent**: Абстрактний клас, який представляє компонент зображення в колажі.
- **ImageLeaf**: Листовий компонент, який представляє одне зображення в колажі.
- **ImageComposite**: Композитний компонент, який представляє колекцію зображень у колажі.

com.example.imageeditor.memento

- **ImageMemento**: Клас Memento для збереження стану зображення.
- **ImageCaretaker**: Клас Caretaker для управління станами зображень та реалізації функціоналу скасування/повторення змін.

com.example.imageeditor.prototype

- **ImagePrototype**: Інтерфейс для клонування зображень.
- **ImageClone**: Реалізація інтерфейсу ImagePrototype для клонування об'єктів зображень.

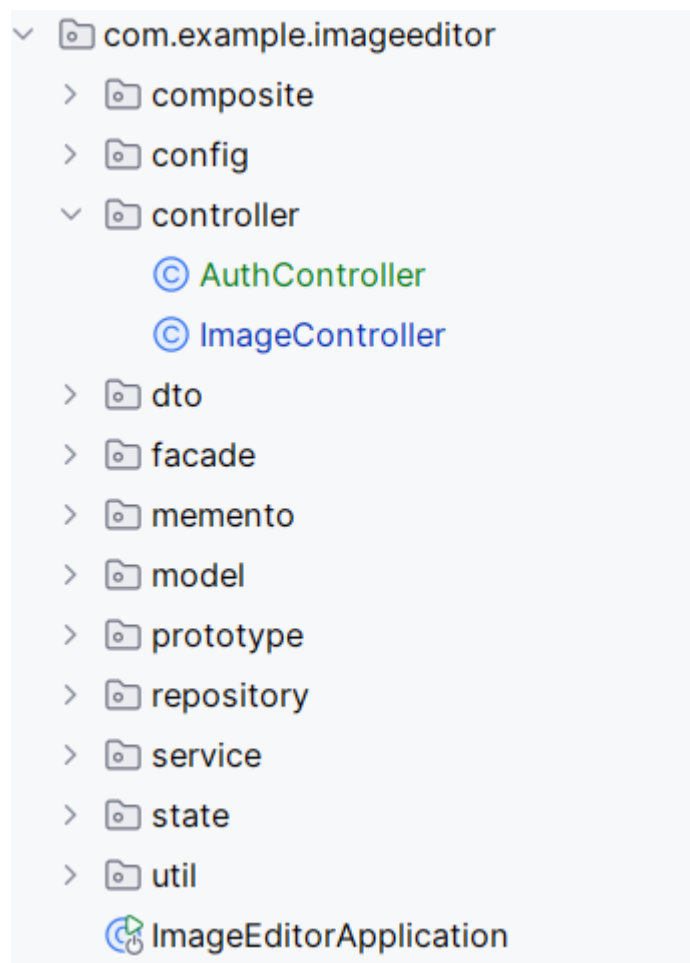


Рис.1 Структура серверної частини

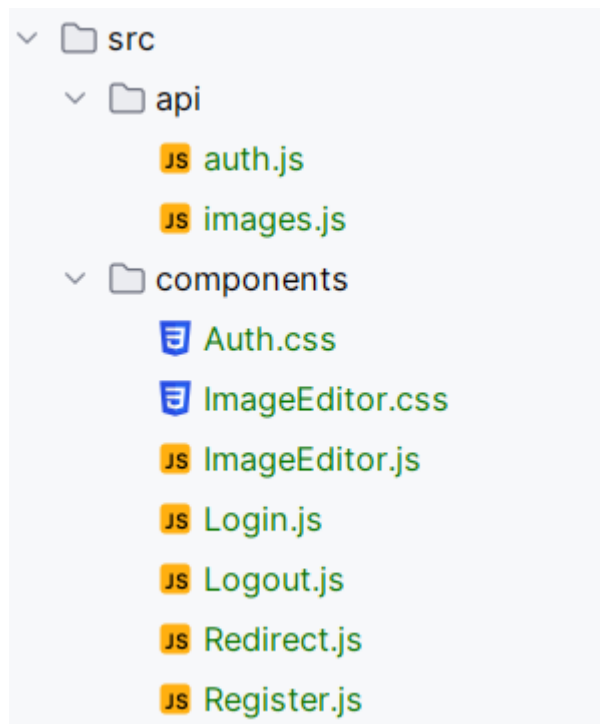


Рис 2. Структура клієнтської частини

Реалізація взаємодії програми в одній з архітектур відповідно до обраної теми.

Клієнт-серверна архітектура

```

@Configuration
public class WebConfig {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return addCorsMappings(registry) → {
            registry.addMapping(pathPattern: "**")
                .allowedOrigins("http://localhost:3000")
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                .allowedHeaders("*")
                .allowCredentials(true);
        };
    }
}
  
```

Рис.3 Конфігурація CORS для серверної частини додатка

```

import Cookies from 'js-cookie';

const API_URL : string = 'http://localhost:8080/api/images';

const getAuthHeader = () : {Authorization: string} => {
  const token = Cookies.get('token');
  if (!token) {
    throw new Error('No authentication token found');
  }
  return {
    'Authorization': `Bearer ${token}`
  };
};

export const uploadImage = async (file) : Promise<Blob> => {
  const formData : FormData = new FormData();
  formData.append( name: 'file', file);

  const response : Response = await fetch( input: `${API_URL}/upload`, init: {
    method: 'POST',
    headers: getAuthHeader(),
    body: formData
  });

  if (!response.ok) {
    throw new Error('Upload failed');
  }

  return response.blob();
};

```

Рис 4. Частина клієнтської логіки

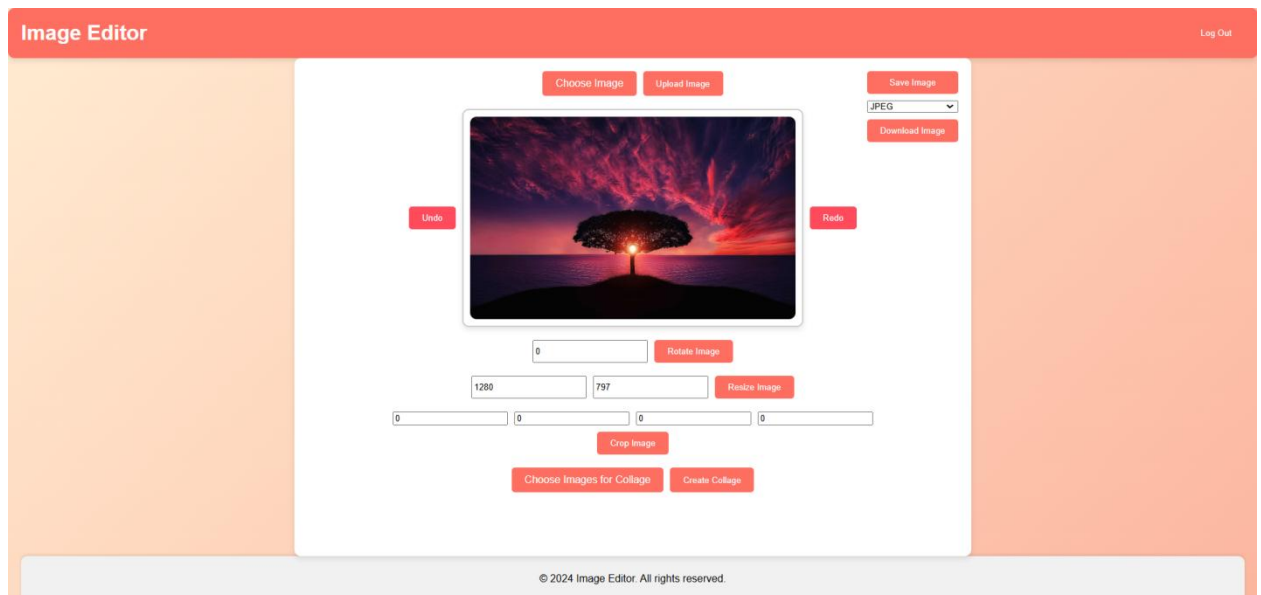


Рис.5 Приклад завантаження зображення на сервер

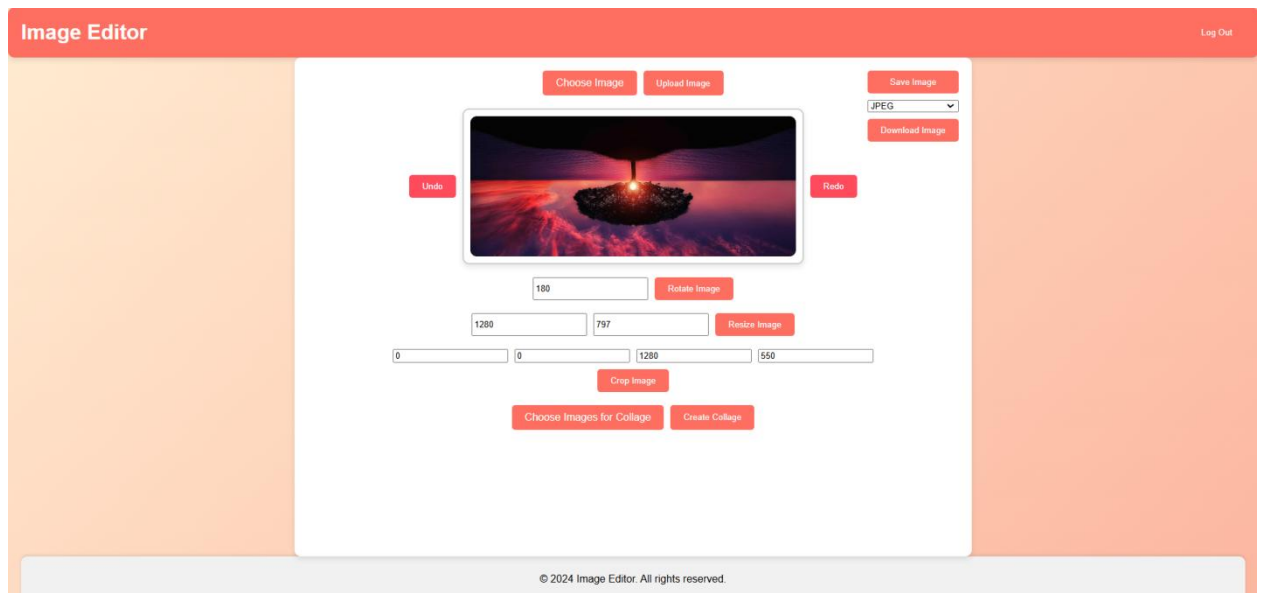


Рис.6 Приклад обертання та обрізання зображення

Висновок: на цій лабораторній роботі я ознайомитися з різними видами взаємодії додатків, а також реалізував частину функціоналу робочої програми у вигляді класів і їх взаємодій для досягнення конкретних функціональних можливостей і реалізував взаємодію програми в клієнт-серверній архітектурі.